

Universidad Mariano Galvez de Guatemala



Curso: Programación III

Tema: Tarea X AVL

Docente: Inge. David Alvarez

Alumno: Carlos Daniel Castro Caceres

main.cpp

```
373 #include <iostream>
375 Usando el espacio de nombres STD;
378
376 Carlos Daniel Castro Cáceres
377 // =====
372 ESTUDIANTE ESTRUCTURA
374 // =====
380
1 ~ struct Student {
379
2     int student_id;
3     cuerda full_name;
381     Carrera de cuerdas;
6     Semestre INT ;
4     GPA de flotación;
10     Int skill_score;
5
8 };
11
7 // =====
15 // NODO AVL
14 // =====
9
13 ~ nodo struct {
12
401     Datos de los estudiantes ;
16     Nodo* izquierda;
398     Nodo* derecha;
404     Altura de Int ;
```

```
399     // =====
404     Altura de Int ;
400
399 };
402
403 // =====
373 // CLASE AVL
375 // =====
378
376 Clase AVL {
377
372 PRIVADO:
374
380     Raíz nodo*;
1     int totalNodes;
379
3 ~ int max(int a, int b) {
2
381     si(a > b) {
6         devolver a;
4     }
10
5     retorno b;
8     }
11
7 ~ int getHeight(Node* node) {
15
14 ~     if(node == NULL) {
9         retorno 0;
13     }
12
```

```

12
401     Retorno >altura del nodo;
16 }
398
404 int getBalance(Node* node) {
399
400     if(node == NULL) {
402         retorno 0;
403     }
373
375     return getHeight(node->left) - getHeight(nodo->right);
378 }
376
377 Nodo* rotarDerecha(Nodo* y) {
372
374     Nodo* x = y->izquierda;
380     Nodo* T2 = x->derecha;
1
379     x->derecha = y;
3     y->izquierda = T2;
381
2     y->altura = max(getHeight(y->izquierda),
6         obtenerAltura(y->derecha)) + 1;
4
10     x->altura = max(getHeight(x->izquierda),
5         getHeight(x->derecha)) + 1;
8
11     Retorno x;
7 }
15

```

```

15
14     Nodo* y = x->derecha;
9     Nodo* T2 = y->izquierda;
13
12     y->izquierda = x;
16     x->derecha = T2;
401
398     x->altura = max(getHeight(x->izquierda),
404         getHeight(x->derecha)) + 1;
399
400     y->altura = max(getHeight(y->izquierda),
402         obtenerAltura(y->derecha)) + 1;
403
375     retorno y;
373 }
376
378 ~
377     Nodo* insertar (Nodo* nodo, estudiante estudiante) {
372 ~
374         if(node == NULL) {
380
381             ~
1         ~
379             newNode->data = estudiante;
3             newNode->left = NULL;
381             newNode->right = NULL;
2             newNode->altura = 1;
6
4             totalNodes++;
10
5             return newNode;

```

```

5         return newNode;
8     }
11
12     Ordenar por skill_score
13     BONUS: desempate por student_id
14
15     if (estudiante.skill_score < nodo->datos.skill_score) {
16
17         nodo->izquierda = insert(nodo->izquierda, estudiante);
18
19     } si no, si (estudiante.skill_score > nodo->datos.skill_score) {
20
21         nodo->derecho = insert(nodo->derecho, estudiante);
22
23     } si no, {
24
25         if (estudiante.student_id < > nodo.student_id) {
26
27             nodo->izquierda = insert(nodo->izquierda, estudiante);
28
29         } si no, {
30
31             nodo->derecho = insert(nodo->derecho, estudiante);
32
33         }
34     }
35
36     nodo->altura = 1 + máximo(getHeight(nodo->izquierda),
37                               getHeight(nodo->right));
38
39     int balance = getBalance(node);

```

```

2     int balance = getBalance(node);
3
4     Izquierda Izquierda
5
6     if (balance > 1 &&
7         estudiante.skill_score < nodo->left->data.skill_score) {
8
9         return rotateRight(node);
10    }
11
12    Eso es cierto
13
14    if (balance < -1 &&
15        estudiante.skill_score > nodo->right->data.skill_score) {
16
17        return rotateLeft(node);
18    }
19
20    Izquierda Derecha
21
22    if (balance > 1 &&
23        estudiante.skill_score > nodo->left->data.skill_score) {
24
25        nodo->izquierda = rotarIzquierda(nodo->izquierda);
26
27        return rotateRight(node);
28    }
29
30    Derecha Izquierda
31

```

```

1
379 ~   if (balance < -1 &&
3 ~   estudiante.skill_score < nodo->right->data.skill_score) {
381
2 ~   nodo->derecho = rotar a la derecha(nodo->derecha);
6
10 ~   return rotateLeft(node);
4 ~   }
5
8 ~   nodo de retorno;
11 ~ }
7
15 ~ void inorder(Nodo* nodo, int & count) {
14
9 ~   if(node != NULL) {
12
13 ~   inorder(node->right, count);
401
16 ~   Conde++;
404
398 ~   Cout << cuenta << ". "
399 ~   << nodo->datos.full_name
400 ~   << " - Puntuación: "
402 ~   << nodo->datos.skill_score
403 ~   << endl;
375
373 ~   inorder(node->left, count);
376 ~   }
378 ~ }
377

```

```

377
372 ~   Nodo* búsqueda(Nodo* nodo, puntuación int ) {
374
380 ~   if(node == NULL ||
379 ~   nodo->datos.skill_score == partitura) {
1
381 ~   nodo de retorno;
3
}
10
6 ~   if (puntuación < nodo->datos.skill_score) {
2
5 ~   return search(node->left, score);
4 ~   }
8
11 ~   return search(node->right, puntuación);
7 ~   }
15
14 Público:
12
9 ~   AVL() {
401
13 ~   raíz = NULL;
404 ~   totalNodes = 0;
16 ~   }
398
399 ~ void insertStudent(Estudiante estudiante) {
400
402 ~   raíz = insert(raíz, estudiante);
403
375 ~   cout << "\nEstudiante insertado correctamente.\n";

```

```

375         cout << "\nEstudiante insertado correctamente.\n";
373     }
376
378     void showRanking() {
377
372         if(root == NULL) {
374
380             cout << "\nNo hay estudiantes registrados.\n";
379
1             Regreso;
3
381         }
6
10         Conteo de inteligencia = 0;
2
4         cout << "\n===== RANKING ACADÉMICO =====\n";
5
8         inorder(raíz, cuenta);
11     }
7
15     void searchStudent(int score) {
14
12         Nodo* resultado = búsqueda(raíz, puntuación);
401
9         if(resultado == NULL) {
13
16             cout << "\nEstudiante no encontrado.\n";
404
398         } si no, {
399
399         cout << "\n===== ESTUDIANTE ENCONTRADO =====\n";

```

```

399         cout << "\n===== ESTUDIANTE ENCONTRADO =====\n";
402
400         cout << "ID: "
403             << resultados->datos.student_id
375             << endl;
373
378         cout << "Nombre: "
376             << resultados->datos.full_name
377             << endl;
372
374         cout << "Carrera: "
380             << resultados->datos.Carrera
379             << endl;
1
3
381         cout << "Semestre: "
10             << resultados->datos.Semestre
6             << endl;
2
4         cout << "GPA: "
5             << resultados->datos.GPA
8             << endl;
11
7         cout << "Puntuación de habilidad: "
12             << resultados->datos.skill_score
14             << endl;
15     }
401
9     void showEstadísticas() {
13

```

```

9 void showEstadísticas() {
13
16     cout << "\n===== ESTADISTICAS AVL =====\n";
404
398     cout << "Altura actual del árbol: "
399     << getHeight(raíz)
402     << endl;
403
400     cout << "Factor de balance de la raíz: "
373     << getBalance (raíz)
375     << endl;
376
378     cout << "Cantidad de nodos insertados: "
377     << totalNodes
372     << endl;
374 }
380 };
379
1 // =====
3 PRINCIPAL
381 // =====
6
10 Int Principal() {
2
4     Árbol AVL ;
5
8     Opción de inteligencia ;
11
7 do {
12

```

```

7 do {
12
14     cout << "\n===== MENÚ AVL =====\n";
15
401     Cout << "1. Insertar estudiante\n";
9     Cout << "2. Mostrar ranking academico\n";
16     Cout << "3. Buscar por skill_score\n";
13     Cout << "4. Mostrar estadísticas AVL\n";
404     Cout << "5. Salir\n";
403
402     cout << "Ingrese una opcion: ";
399     Opción >> CIN;
398
373 switch(option)) {
400
375     Caso 1: {
376
378         Estudiantes ;
372
377         cout << "\nIngrese ID: ";
374         Cin >> S.student_id;
379
380         cin.ignorar();
1
3         cout << "Ingrese nombre completo: ";
381         Getline (Cin, S.full_name);
6
10         cout << "Ingrese carrera: ";
2         Getline (Cin, S.Carrera);
4

```

```

4
5         cout << "Ingreso semestre: ";
8         Cin >> S.Semestre;
11
12         cout << "GPA de Ingreso: ";
13         Cin >> S.GPA;
14
15         cout << "Puntuación de habilidad Ingreso: ";
401        Cin >> S.skill_score;
9
16         árbol.insertStudent(s));
13
404         Pausa;
402     }
403
399 ~     Caso 2: {
398
373         árbol.showRanking();
400
375         Pausa;
376     }
378
372 ~     Caso 3: {
377
374         Puntuación de Int ;
379
380         cout << "\nIngreso el score a buscar: ";
1         Cin >> partitura;
3
381         árbol.searchStudent(score);

```

```

381         árbol.searchStudent(score);
6
10         Pausa;
2     }
4
5 ~     Caso 4: {
8
11         árbol.mostrarEstadísticas();
12
13         Pausa;
14     }
401 ~     Caso 5: {
9
16         cout << "\nSaliendo del programa...\n";
13
404         Pausa;
402     }
403
399 ~     Por defecto: {
373
398         cout << "\nOpcion invalida.\n";
400     }
375 }
376
378 } while(opción!= 5);
372
377 retorno 0;
374 }

```



```
===== MENU AVL =====
1. Insertar estudiante
2. Mostrar ranking academico
3. Buscar por skill_score
4. Mostrar estadísticas AVL
5. Salir
Ingrese una opcion: 1

Ingrese ID: 1
Ingrese nombre completo: Carlos Daniel Castro Caceres
Ingrese carrera: Ingenieria en sistemas
Ingrese semestre: 5
Ingrese GPA: 90
Ingrese skill score: 95
```

Estudiante insertado correctamente.

```
===== MENU AVL =====
1. Insertar estudiante
2. Mostrar ranking academico
3. Buscar por skill_score
4. Mostrar estadísticas AVL
5. Salir
Ingrese una opcion: 1

Ingrese ID: 2
Ingrese nombre completo: Juan David Osorio Pascual
Ingrese carrera: Ingenieria en sistemas
Ingrese semestre: 5
Ingrese GPA: 95
Ingrese skill score: 120
```

Ingrese skill score: 120

Estudiante insertado correctamente.

```
===== MENU AVL =====
1. Insertar estudiante
2. Mostrar ranking academico
3. Buscar por skill_score
4. Mostrar estadísticas AVL
5. Salir
Ingrese una opcion: 1

Ingrese ID: 2
Ingrese nombre completo: Jose Manuel Arroyo Lopez
Ingrese carrera: Ingenieria en sistemas
Ingrese semestre: 6
Ingrese GPA: 98
Ingrese skill score: 200
```

Estudiante insertado correctamente.

```
===== MENU AVL =====
1. Insertar estudiante
2. Mostrar ranking academico
3. Buscar por skill_score
4. Mostrar estadísticas AVL
5. Salir
Ingrese una opcion: 2
```

```
===== RANKING ACADEMICO =====
1. Jose Manuel Arroyo Lopez - Score: 200
2. Juan David Osorio Pascual - Score: 120
3. Carlos Daniel Castro Caceres - Score: 95
```

3. Carlos Danieel Castro Caceres - Score: 95

===== MENU AVL =====

1. Insertar estudiante
2. Mostrar ranking academico
3. Buscar por skill_score
4. Mostrar estadísticas AVL
5. Salir

Ingrese una opcion: 3

Ingrese el score a buscar: 95

===== ESTUDIANTE ENCONTRADO =====

ID: 1

Nombre: Carlos Danieel Castro Caceres

Carrera: Ingenieria en sistemas

Semestre: 5

GPA: 90

Skill Score: 95

===== MENU AVL =====

1. Insertar estudiante
2. Mostrar ranking academico
3. Buscar por skill_score
4. Mostrar estadísticas AVL
5. Salir

Ingrese una opcion: 4

===== ESTADISTICAS AVL =====

Altura actual del arbol: 2

Factor de balance de la raiz: 0

Cantidad de nodos insertados: 3